

1P-G3-IATI-resuelto.pdf



andreagb03



Inteligencia Artificial



3º Grado en Ingeniería Informática - Tecnologías Informáticas



Escuela Técnica Superior de Ingeniería Informática
Universidad de Sevilla



[Accede al documento original](#)

INTERCAMBIA y AHORRA en LA VUELTA A LA UNI

5 AÑOS DE GARANTÍA en todo

webuy.com



SUSCRIBIRME

En plan **Hot**

Movistar Plus: 4,99.

Sobresaliente en entretenimiento.



PLAN
LIBRE | **4,99€**
MES

27/12/24, 12:14

1P G3 IATI resuelto

Inteligencia Artificial

Parte práctica de la primera prueba de evaluación alternativa. Curso 2022-23

Grupo 3

In [7]: `# Apellidos:`
`# Nombre:`

En los siguientes ejercicios vamos a implementar algunas funciones relacionadas con los proceso de decisión de Markov. Definimos un proceso de decisión de Markov como una lista $[R, T]$ donde R es una lista de pares (*estado, recompensa*) y T es el modelo de transición. El modelo de transición contiene listas $[s, a, (s_1, p_1), \dots, (s_n, p_n)]$ donde $s_1, \dots, s_n$ son los estados a los que se puede acceder desde s y p_i representa la probabilidad de llegar a s_i desde s realizando la acción a . Algunos ejemplos son:

```
In [3]: MDP_1 = [[('s1', -0.05), ('s2', -0.05), ('s3', -1), ('s4', 1)],
                [['s1', 'a1', ('s1', 0.1), ('s2', 0.8), ('s3', 0.1)],
                 ['s1', 'a2', ('s2', 0.3), ('s3', 0.7)],
                 ['s2', 'a1', ('s1', 0.3), ('s2', 0.1), ('s3', 0.1), ('s4', 0.5)],
                 ['s2', 'a2', ('s2', 0.6), ('s4', 0.4)],
                 ['s2', 'a3', ('s1', 0.9), ('s4', 0.1)],
                 ['s2', 'a4', ('s1', 0.8), ('s2', 0.2)],
                 ['s3', 'a1', ('s2', 0.1), ('s3', 0.1), ('s4', 0.8)],
                 ['s3', 'a2', ('s1', 0.3), ('s4', 0.7)],
                 ['s3', 'a4', ('s1', 0.6), ('s2', 0.1), ('s3', 0.1), ('s4', 0.2)],
                 ['s4', 'a1', ('s1', 0.3), ('s2', 0.1), ('s3', 0.1), ('s4', 0.7)],
                 ['s4', 'a2', ('s1', 0.3), ('s4', 0.7)],
                 ['s4', 'a4', ('s1', 0.2), ('s3', 0.1), ('s4', 0.7)]]]

MDP_2 = [[('e1', -0.3), ('e2', -0.1), ('e3', 2)],
          [['e1', 'acc_1', ('e1', 0.1), ('e2', 0.9)],
           ['e1', 'acc_2', ('e2', 0.3), ('e3', 0.7)],
           ['e1', 'acc_3', ('e1', 0.3), ('e2', 0.1), ('e3', 0.6)],
           ['e2', 'acc_1', ('e2', 0.6), ('e3', 0.4)],
           ['e2', 'acc_2', ('e1', 1)],
           ['e3', 'acc_1', ('e1', 0.8), ('e2', 0.2)],
           ['e3', 'acc_3', ('e1', 0.1), ('e3', 0.9)],
           ['e3', 'acc_4', ('e1', 0.3), ('e2', 0.7)]]]
```

Representaremos una política como una lista de dos listas. La primera de ella representa los estados y la segunda las acciones asociadas a esos estados.

```
In [6]: pi_1 = [['s1', 's2', 's3', 's4'], ['a1', 'a1', 'a1', 'a1']]
pi_2 = [['s1', 's2', 's3', 's4'], ['a2', 'a3', 'a4', 'a2']]

pi_3 = [['e1', 'e2', 'e3'], ['acc_2', 'acc_1', 'acc_4']]
pi_4 = [['e1', 'e2', 'e3'], ['acc_1', 'acc_2', 'acc_3']]
```

WUOLAH

Nota: En los siguientes ejercicios, el estudiante puede añadir y/o modificar el código que considere necesario, más allá de completar los huecos indicados para ello.

Ejercicio 1 (0,5 ptos):

Definir una función `accion_recomendada(s,pi)` que reciba como dato de entrada un estado s y una política pi y devuelva la acción recomendada para dicho estado s según la política pi .

Por ejemplo, si consideramos el estado s_1 y la política pi_1 la acción recomendada sería la acción a_1 . Si consideramos el estado e_1 y la política pi_3 la acción recomendada sería la acción acc_2

```
In [10]: # Solución:
def accion_recomendada(s,pi):
    estados,acciones = pi
    if s in estados:
        indice = estados.index(s)
        return acciones[indice]
    else :
        return None
    #???
```

```
In [12]: accion_recomendada("s1",pi_1)
# Respuesta esperada: "a1"
```

```
Out[12]: 'a1'
```

```
In [14]: accion_recomendada("s4",pi_2)
# Respuesta esperada: "a2"
```

```
Out[14]: 'a2'
```

```
In [16]: accion_recomendada("e1",pi_3)
# Respuesta esperada: "acc_2"
```

```
Out[16]: 'acc_2'
```

```
In [18]: accion_recomendada("e1",pi_4)
# Respuesta esperada: "acc_1"
```

```
Out[18]: 'acc_1'
```

Ejercicio 2 (0.5 ptos)

Define una función `ep(MDP,s,pi)` que tome como entrada un proceso de decisión de Markov, un estado s y una política pi y devuelva una lista de pares $(s',prob)$ donde los s' son los estados a los que se puede llegar desde s tras aplicar la acción que determina la política pi y prob es la probabilidad de llegar a ese estado. Por ejemplo si tomamos el proceso de decisión MDP_1, el estado s_2 y la política pi_2 , vemos que la acción recomendada para s_2 por esa política es a_3 y que si aplicamos la acción a_3 a s_2 llegamos a

- s1 con probabilidad 0.9
- s4 con probabilidad 0.1

Por tanto la salida de la función debe ser [('s1', 0.9), ('s4', 0.1)]

```
In [20]: # Solución:
def ep(MDP,s,pi):
    _, transiciones = MDP#Desempaquetamos MDP en sus componentes recompensas y t
    accion=accion_recomendada(s,pi)
    for transicion in transiciones:
        if transicion[0] == s and transicion[1] == accion:#Si transicion corresp
            return transicion [2:]#extraemos los estados alcanzables y las proba
    return []
    #????
```

```
In [22]: ep(MDP_1,'s2',pi_2)
# Respuesta esperada: [('s1', 0.9), ('s4', 0.1)]
```

```
Out[22]: [('s1', 0.9), ('s4', 0.1)]
```

```
In [24]: ep(MDP_1,'s4',pi_1)
# Respuesta esperada: [('s1', 0.3), ('s2', 0.1), ('s3', 0.1), ('s4', 0.7)]
```

```
Out[24]: [('s1', 0.3), ('s2', 0.1), ('s3', 0.1), ('s4', 0.7)]
```

```
In [26]: ep(MDP_2,'e1',pi_3)
# Respuesta esperada: [('e2', 0.3), ('e3', 0.7)]
```

```
Out[26]: [('e2', 0.3), ('e3', 0.7)]
```

```
In [28]: ep(MDP_2,'e2',pi_4)
# Respuesta esperada: [('e1', 1)]
```

```
Out[28]: [('e1', 1)]
```

Ejercicio 3 (0.75 pto):

Definir una función `valoracion_secuencia(MDP,sec,desc)` que reciba como dato de entrada un proceso de decisión de Markov *MDP*, una secuencia de estados *sec* y un factor de descuento *desc* y devuelva la valoración de dicha secuencia de estados:

$$V([q_0q_1q_2\dots]) = R(q_0) + \gamma R(q_1) + \gamma^2 R(q_2) + \dots$$

Por ejemplo, si consideramos el problema *MDP_1*, la secuencia s_1, s_2, s_3, s_4 y un factor de descuento 0.5 el resultado de la valoración sería:

$$V([s_1, s_2, s_3, s_4]) = R(s_1) + \gamma R(s_2) + \gamma^2 R(s_3) + \gamma^3 R(s_4) = -0.05 + 0.5(-0.05) + 0.5^2(-0.05) + 0.5^3(-0.05)$$

```
In [41]: # Solución
def valoracion_secuencia(MDP,sec,desc):
    v=0 #=????
    for idx,s in enumerate(sec):#in ??? #Recorremos la secuencia con indices
        for sp in MDP[0]: #iteramos sobre la lista de recompensas en MDP
```



SUSCRIBIRME

En plan Hot
Movistar Plus: 4,99.
Sobresaliente en entretenimiento.



PLAN
LIBRE | **4,99€**
MES

27/12/24, 12:14

1P G3 IATI resuelto

```
if sp[0]==s: #==???? #si el estado coincide con el de la secuencia
    v = v + (desc **idx) * sp[1] #+???? #añadimos la recompensa por
    break
return v
```

In [43]: `valoracion_secuencia(MDP_1,['s1','s2','s3','s4'],0.5)`
Respuesta esperada: -0.2

Out[43]: -0.2

In [45]: `valoracion_secuencia(MDP_1,['s1','s2','s1','s2'],0.25)`
Respuesta esperada: -0.06640625

Out[45]: -0.06640625

In [47]: `valoracion_secuencia(MDP_2,['e1','e2','e2'],0.5)`
Respuesta esperada: -0.375

Out[47]: -0.375

In [49]: `valoracion_secuencia(MDP_2,['e2','e2','e2'],0.9)`
Respuesta esperada: -0.271

Out[49]: -0.271

Ejercicio 4 (0.75 pts):

Definir una función `probabilidad_secuencia(MDP,sec,pi)` que reciba como dato de entrada un proceso de decisión de Markov *MDP*, una secuencia de estados *sec* y una política *pi* y devuelva la probabilidad de dicha secuencia de estados bajo la política *pi*.

Por ejemplo, si consideramos el problema *MDP_1*, la secuencia $[s_1, s_2, s_3, s_4]$ y la política π_1 (que asigna la acción a_1 a todos los estados), la probabilidad resultante sería:

$$0.8 \times 0.1 \times 0.8 = 0.064$$

Ya que 0.8 es la probabilidad de pasar al estado s_2 desde el estado s_1 aplicando la acción recomendada por la política π_1 para s_1 , 0.1 es la probabilidad de pasar al estado s_3 desde el estado s_2 aplicando la acción recomendada por la política π_1 para s_2 y 0.8 es la probabilidad de pasar al estado s_4 desde el estado s_3 aplicando la acción recomendada por la política π_1 para s_3 .

Nota: Se recomienda el uso de la función `ep(MDP,s,pi)` definida en el Ejercicio 1.

```
In [30]: # Solución:
def probabilidad_secuencia(MDP,sec,pi):
    p=1 #?????
    for idx,s in enumerate(sec[:-1]):
        transiciones=ep(MDP,s,pi) #?????
        encontrado = False
        for t in transiciones:
            if t[0]==sec[idx+1]: #?????
                encontrado = True
            p = p * t[1] #?????
```



```

    if not encontrado:
        p=0 #=????
    return p

```

```

In [32]: probabilidad_secuencia(MDP_1,["s1","s2","s3","s4"],pi_1)
# Salida esperada: 0.06400000000000002

```

```

Out[32]: 0.06400000000000002

```

```

In [34]: probabilidad_secuencia(MDP_1,["s1","s1","s1","s1"],pi_2)
# Salida esperada: 0

```

```

Out[34]: 0

```

```

In [36]: probabilidad_secuencia(MDP_2,["e1","e1","e1","e1"],pi_3)
# Salida esperada: 0

```

```

Out[36]: 0

```

```

In [38]: probabilidad_secuencia(MDP_2,["e1","e1","e1","e1"],pi_4)
# Salida esperada: 0.0010000000000000002

```

```

Out[38]: 0.0010000000000000002

```

Ejercicio 5 (1 pto):

Definir una función `valoracion_respecto_politica(MDP,pi,descuento,k)` que aplica el método de iteración de valores *con una política fija* (k iteraciones) para calcular la valoración V^π (dicha valoración debe devolverse como un diccionario que a cada estado s le asocia el valor $V^\pi(s)$ calculado). El algoritmo se describe en el punto 2 de la diapositiva 18 del tema 4.

```

In [40]: # Solución:

def valoracion_respecto_politica(MDP,pi,descuento,k):

    Rdict={estado: recompensa for estado,recompensa in MDP[0]} #=????
    # consejo: crear la función R como un diccionario que
    # a cada estado le asigna su recompensa

    estados=[estado for estado,_ in MDP[0]] # Lista de estados

    V = {e:0 for e in estados}
    for _ in range(k):
        V1 = V.copy()
        for s in estados:
            accion = accion_recomendada(s,pi)
            valor_futuro = 0
            for s_prima,prob in ep(MDP,s,pi):
                valor_futuro += prob * V1[s_prima]
            V[s] = Rdict[s] + descuento * valor_futuro
    return V

```

Ejemplos:

DALE AMOR A TU CUERPO, DALE AMOR A TU COCO



Hasta el 15 de nov.

DESCUENTO EXTRA

3€
MES

EN CUOTA
PRIME

con el código

VGWUOLAH26



inscríbete

*Promo válida para altas desde el 01/10/2026 hasta el 15/11/2026.
Código personal e intransferible. Acumulable con otras promociones.
Descuento de 3€ en Cuota Prime durante 3 meses. Cuota posterior,
oficial del club. Promo válida en todos los clubs de España excepto en
los clubs en preopening.

VIVAGYM

In [43]: `valoracion_respecto_politica(MDP_1,pi_1,0.8,100)`

```
# Salida esperada (aproximada)
# {'s1': 1.6701705266033209,
#  's2': 2.24891713655786,
#  's3': 1.8406239634753088,
#  's4': 3.9272823054260235}
```

Out[43]: {'s1': 1.6701664783639991,
's2': 2.248912705576413,
's3': 1.8406191559618197,
's4': 3.9272768836079415}

In [45]: `valoracion_respecto_politica(MDP_1,pi_2,0.7,200)`

```
# Salida esperada (aproximada):
# {'s1': -0.7012402156446009,
#  's2': -0.37473865187565863,
#  's3': -1.168459385205536,
#  's4': 1.6720383425777134}
```

Out[45]: {'s1': -0.7012402156446009,
's2': -0.37473865187565863,
's3': -1.168459385205536,
's4': 1.6720383425777134}

In [47]: `valoracion_respecto_politica(MDP_2,pi_3,0.8,200)`

```
# Salida esperada (aproximada):
# {'e1': 2.2908745247148286, 'e2': 2.100760456273764, 'e3': 3.7262357414448664}
```

Out[47]: {'e1': 2.2908745247148286, 'e2': 2.100760456273764, 'e3': 3.7262357414448664}

In [49]: `valoracion_respecto_politica(MDP_2,pi_4,0.9,500)`

```
# Salida esperada (aproximada):
# {'e1': -2.1049178548875442, 'e2': -1.9944206173964403, 'e3': 9.529278124110025}
```

Out[49]: {'e1': -2.1049723756906076, 'e2': -1.9944751381215469, 'e3': 9.529223611514983}